

이해를 향한 여정 - 인간의 기억에서 RNN/LSTM 까지

1 부: 사람의 기억을 기계로 이해하기



perplexity



Claude

Gemini

- 최근 **ChatGPT, Gemini, Claude** 같은 AI 가 빠르게 확산되며 우리의 일상 속에 깊숙이 자리 잡고 있습니다.
- AI 를 활용하면
복잡하고 시간이 오래 걸리던 일도 훨씬 빠르게 처리할 수 있고,
업무와 학습의 효율을 크게 높여줍니다.
- 그렇다면 이렇게 우리 삶에 큰 변화를 가져온 AI, 특히 ChatGPT 같은 대형 언어모델(LLM)은 어디서부터 시작된 걸까요?

1.1 정보 처리 모델: 우리 머릿속의 컴퓨터

뇌와 컴퓨터의 비유 (컴퓨터 유비)

- 옛날 연구자들은 인간의 기억과 사고 과정을 설명할 때, **컴퓨터 작동 방식**에 자주 빗대어 설명했습니다.

입력(Input): 사람이 감각으로 정보를 받아들이듯, 컴퓨터도 키보드/센서로 데이터를 받음

저장(Storage): 사람은 단기/장기 기억이 있고, 컴퓨터는 RAM/하드디스크가 있음

출력(Output): 사람이 학습 결과를 행동으로 보이듯, 컴퓨터는 계산된 결과를 보여줌

→ 이 때문에 "뇌 = 일종의 컴퓨터"라는 생각이 한때 널리 퍼졌습니다.

하지만...

1.2 문제점

뇌는 컴퓨터와 다릅니다.

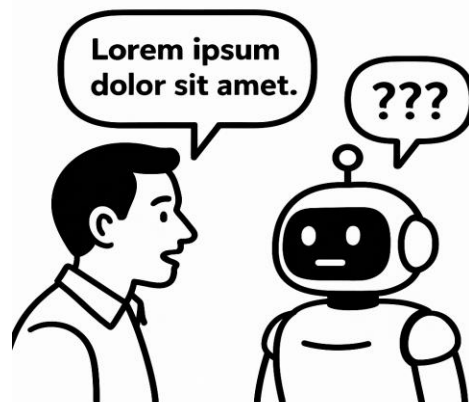
- 뇌는 **느리고 병렬적** (여러 일을 동시에 처리), 컴퓨터는 **빠르고 직렬적** (한 번에 한 과정)
- 뇌는 감정, 동기, 경험에 영향을 받지만 컴퓨터는 그렇지 않음
- 기억도 "폴더 저장"처럼 정확히 보관되는 게 아니라, 매번 **재구성**됨

가장 큰 차이점 중 하나는 **처리 방식**입니다. 인간의 뇌는 광범위한 병렬 처리에 매우 능숙합니다. 반면, 전통적인 컴퓨터는 직렬 처리에 의존하는 경우가 많습니다. 예를 들어, 숙련된 타이피스트는 여러 글자를 미리 생각하며 (병렬적으로) 타이핑하지만, 초보자는 한 번에 한 글자에만 집중합니다 (직렬적으로).

또한, **생물학과 감정의 역할**을 간과할 수 없습니다. 뇌는 정보를 전달하기 위해 화학 물질을 사용하는 반면, 컴퓨터는 순전히 전기 신호를 사용합니다. 이는 뇌의 신호 전달이 컴퓨터보다 본질적으로 느리다는 것을 의미합니다. 더 중요한 것은, 인간의 인지는 상충하는 감정적, 동기적 요인에 깊은 영향을 받는다는 점입니다. 우리는 단순히 정보를 저장하는 것이 아니라, 직업을 유지하거나, 승진하거나, 자부심을 느끼기 위해 학습합니다.

기억은 하드 드라이브가 아니라는 점도 중요합니다. 인간의 기억은 파일처럼 완벽하게 저장되고 인출되는 것이 아닙니다. 오히려 기억은 재구성되며, 다른 기억과의 연결, 인식된 가치, 인출 빈도에 따라 변화할 수 있습니다. 컴퓨터는 기호적 표현을 완벽하게 저장하고 검색하지만, 유기체는 그렇지 않습니다.

1.3 위대한 도전: 인간과 기계의 언어 격차 해소하기



인간의 마음과 컴퓨터가 이토록 다르다면, 특히 언어처럼 미묘하고 맥락이 풍부한 정보를 처리하는 방식에서 근본적인 차이가 있다면, 우리는 어떻게 기계에게 우리의 말을 이해하도록 가르칠 수 있을까요?

이 질문은 바로 **자연어 처리(Natural Language Processing, NLP)**라는 학문 및 공학 분야 전체의 출발점이 됩니다. 예를 들어, 기억이 단순한 데이터 인출 과정이 아니라는 깨달음은 장거리 의존성과 문맥적 관계를 처리할 수 있는 모델의 필요성으로 직접 이어졌고, 이는 순환 신경망(RNN)과 LSTM의 개발을 촉발했습니다. 이 학문이 발전해 지금의 AI가 나오게 되었습니다.

2 부: 기계 대화의 역사: NLP 의 진화

2.1 분야 정의: 자연어 처리(NLP)란 무엇인가?

자연어 처리(NLP)는 인공지능(AI)의 한 분야로, 컴퓨터가 인간의 언어를 이해하고, 생성하며, 조작할 수 있도록 하는 기술입니다. 그 목표는 컴퓨터가 인간이 말하고 쓰는 그대로의 언어를 이해하도록 프로그래밍하는 것입니다. NLP 는 텍스트와 음성이라는 비정형 데이터를 분석하여 구조를 부여하고, 이를 통해 기계가 언어를 이해할 수 있게 합니다.

NLP 가 실제로 수행하는 핵심 작업들은 다음과 같이 나눌 수 있습니다.

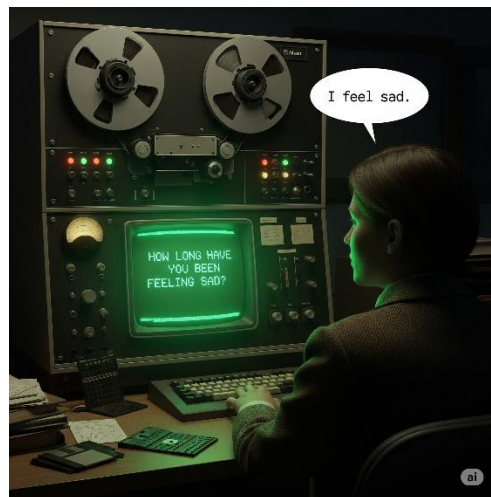
- **구문 분석(Syntax Analysis):** 문장의 문법적 구조를 이해하는 과정입니다. 품사 태깅(명사, 동사, 형용사 등을 식별)이나 문장 구조 분석 등이 여기에 해당합니다.
- **의미 분석(Semantic Analysis):** 텍스트로부터 의미를 도출하는 과정입니다. 단어의 중의성을 해소하거나(word-sense disambiguation), 문장 내 개체들 간의 관계를 파악하는 작업이 포함됩니다.
- **화용 분석(Pragmatic Analysis):** 주어진 맥락을 바탕으로 텍스트의 실제 목적이나 의도를 이해하는 것입니다.

이러한 기술들은 챗봇, 검색 엔진, 스팸 메일 필터링, 감성 분석, 기계 번역 등 우리 일상 속 다양한 응용 프로그램의 기반이 됩니다.⁶

2.2 NLP 의 역사 (1950 - 1970)

NLP 역사의 첫 번째 주요 패러다임은 규칙 기반 접근법이었습니다. 이 시기 초기 NLP 시스템들은 언어학자와 프로그래머들이 직접 손으로 복잡한 문법 규칙들을 코딩하여 컴퓨터가 이를 따르도록 만드는 방식에 의존했습니다. 이는 노암 촘스키의 생성 문법과 같은 초기 언어학 이론에서 영감을 받았습니다.

대표적인 사례로 ELIZA (1964-1966)를 들 수 있습니다. ELIZA 는 정신과 의사와의 대화를 시뮬레이션하는 프로그램으로, 사용자의 입력을 특정 패턴과 일치시키고 문장 구조를 재배열하여 질문을 생성하는 방식으로 작동했습니다. 예를 들어, 사용자가 "나는 슬픔을 느껴요"라고 말하면, ELIZA 는 "얼마나 오랫동안 슬픔을 느끼셨나요?"와 같이 응답할 수 있었습니다.



하지만 이 접근법의 근본적인 한계는 시스템이 언어를 **진정으로 이해하지 못했다는** 점입니다. 이 방식은 매우 취약해서, 사전에 코딩되지 않은 속어나 오타, 새로운 문법 구조를 만나면 쉽게 실패했습니다.

2.3 통계적 전환 (1980 - 2000)

규칙 기반 시스템의 취약성에 좌절한 연구자들은 1980 년대와 90 년대에 통계적 방법으로 눈을

돌렸습니다. 이는 규칙을 프로그래밍하는 대신, 방대한 양의 텍스트 데이터(코퍼스, corpus)로부터 시스템이 직접 패턴을 학습하도록 하는 패러다임의 전환이었습니다. 핵심 아이디어는 규칙에 기반한 경직된 결정이 아닌, 확률에 기반한 유연한 결정을 내리는 것이었습니다. 이 시대를 대표하는 통계 모델이 바로 **N-gram 모델**입니다.

N-gram 모델이란?

- 문장에서 연속된 **N 개의 단어 묶음**(토큰)을 보고, 다음 단어가 무엇일지 확률적으로 예측하는 모델.
- 규칙을 직접 짜지 않고, **데이터(코퍼스) 속 패턴**을 통계적으로 학습하는 접근.

N-gram 은 텍스트에서 연속적으로 나타나는 'N'개의 단어 묶음을 의미합니다.

- **유니그램 (Unigram, 1-gram)**: "the", "cat"과 같은 개별 단어.
- **바이그램 (Bigram, 2-gram)**: "the cat"과 같은 단어 쌍.
- **트라이그램 (Trigram, 3-gram)**: "the cat sat"과 같은 세 단어 묶음.

N-gram 모델은 이전 N-1 개의 단어가 주어졌을 때 다음 단어가 나타날 확률을 계산합니다.

예를 들어, $P(\text{"sat"} \mid \text{"the cat"})$ 의 확률은 훈련 데이터에서 "the cat"이 나타난 횟수 대비 "the cat sat"이 나타난 횟수를 세어 계산합니다. 이는 오늘날 자동 완성 기능의 기초가 되는 원리입니다.

N-gram 의 약점

하지만 N-gram 모델의 강점인 단순함은 동시에 약점이기도 했습니다. 바로 **제한된 문맥**입니다. 트라이그램 모델은 다음 단어를 예측할 때 바로 앞의 두 단어 외에는 아무것도 기억하지 못합니다.

예시)

“그 학생은 매일 아침 학교에 갑니다.”

- **Bigram (2-gram)**
- “학교에 → ?”

- 그러면 "갑니다"를 고를 수도 있지만, "있습니다", "다닙니다" 같은 것도 확률 상 나올 수 있습니다.
- 문제는 "학생은"이라는 주어 정보는 이미 기억이 안 나서, **문법적으로 정확한 답을** 보장하지 못합니다.

이는 **데이터 희소성(data sparsity)** 문제로도 이어집니다. N 이 커질수록 가능한 N-gram 의 조합은 기하급수적으로 늘어나고, 대부분의 조합은 훈련 데이터에 한 번도 나타나지 않아 확률이 0 이 되는 문제가 발생합니다.

이러한 NLP 의 역사는 '취약함(brittle)'과 '모호함(blurry)' 사이의 끊임없는 긴장 관계로 볼 수 있습니다. 규칙 기반 시스템은 정밀했지만 예상치 못한 입력에 쉽게 무너지는 '취약함'을 가졌습니다.

규칙 기반보다 유연하고 현실 언어를 잘 처리했지만, **짧은 기억 + 데이터 부족 문제** 때문에 한계가 컸음.

이 한계가 결국 "더 긴 문맥을 기억할 수 있는 모델(RNN)의 필요성을 낳았습니다.

3 부: 기억의 구조: 순환 신경망(RNN)

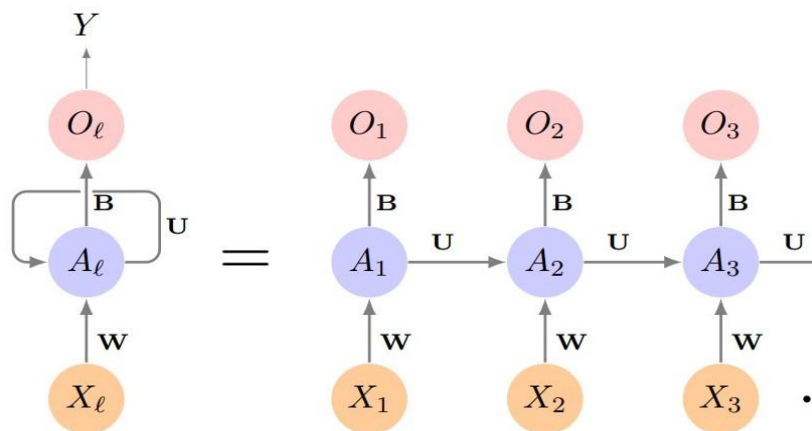
3.1 N-gram 의 한계

N-gram 모델의 핵심적인 실패는 작고 고정된 창(window) 너머의 정보를 기억하지 못한다는

것이었습니다. 언어를 제대로 이해하기 위해서는 문장과 같은 순차적인 데이터를 한 번에 한 단어씩 처리하면서도 이전에 본 내용을 어떤 형태로든 기억할 수 있는 모델이 필요했습니다. 즉, 모델 자체에 일종의 기억 장치가 필요했던 것입니다.

3.2 기억의 고리: 순환 신경망(RNN)의 등장

순환 신경망(Recurrent Neural Network, RNN)은 바로 이러한 순차적 데이터를 위해 특별히 설계된 신경망 유형입니다. RNN의 가장 결정적인 특징은 정보가 지속될 수 있도록 하는 '루프(loop)' 또는 '순환' 구조에 있습니다.¹⁷



구조를 간단히 말하면:

1. 현재 단어(입력)를 받음
2. 이전까지의 은닉 상태(기억)와 합쳐서
3. 새로운 **출력** + 새로운 **은닉 상태**를 만듦

RNN의 구조를 간단히 살펴보면 다음과 같습니다. 각 타임스텝(time step, 예를 들어 문장의 각

단어)에서 RNN 은 두 가지 입력을 받습니다.

1. 현재 타임스텝의 입력 데이터 (예: 'cat'이라는 단어의 벡터 표현)
2. 이전 타임스텝으로부터 전달된 **은닉 상태(hidden state)**

여기서 **은닉 상태**가 핵심 개념입니다. 은닉 상태는 숫자로 이루어진 벡터로, 네트워크의 **기억** 역할을 합니다. 이 벡터에는 해당 시점까지 입력된 모든 순서열의 정보를 압축한 요약본이 담겨 있습니다.

이는 우리가 문장을 읽는 과정과 유사합니다. 글을 읽을 때 우리는 현재 단어만 처리하는 것이 아니라, 이전에 읽었던 단어들이 형성한 문맥 속에서 현재 단어를 이해합니다. RNN 의 은닉 상태는 마치 문장을 따라가며 점진적으로 형성되는 우리의 이해 과정과 같습니다.

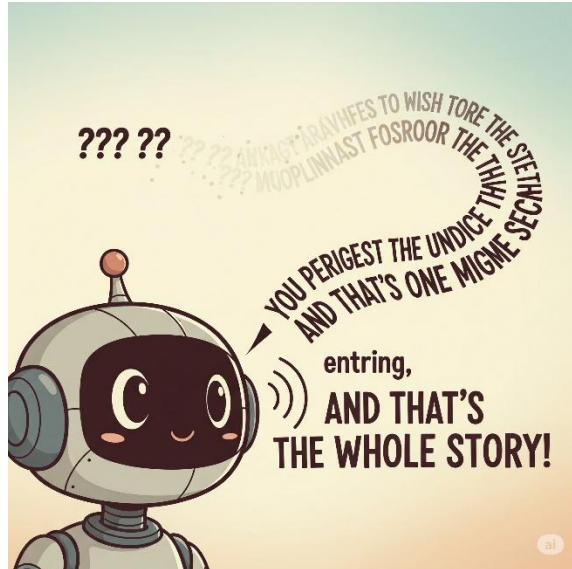
3.3 희미해지는 메아리: 기울기 소실 문제

RNN 은 이론적으로는 아주 오래전의 정보도 기억할 수 있지만, **기울기 소실 문제**(vanishing gradient problem)라는 문제 때문에 잘 되지 않습니다.

신경망을 훈련시키는 과정은 예측이 틀렸을 때 발생하는 오차(error)를 기반으로 내부 가중치(weight)를 조정하는 것을 포함합니다. 이 오차 신호, 즉 '기울기(gradient)'는 순서열의 끝에서부터 시작까지 네트워크를 거꾸로 전파되어야 합니다(이를 BPTT, Backpropagation Through Time 이라고 합니다).

기울기가 순환을 거칠 때마다 작은 숫자들과 반복적으로 곱해집니다. 이로 인해 신호는 기하급수적으로 작아져, 후반부에 도착할 땐 거의 '소실'되어 버립니다.

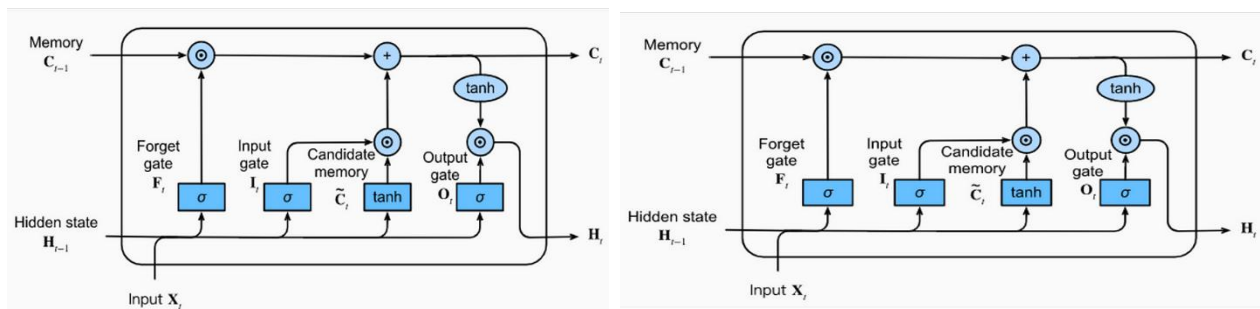
그 결과, 네트워크는 긴 문장의 시작 부분에 있는 단어와 끝 부분에 있는 단어 사이의 연관성을 학습할 수 없게 됩니다. 기억이 너무 단기적이어서 초기 정보의 메아리가 너무 빨리 사라지는 것입니다.



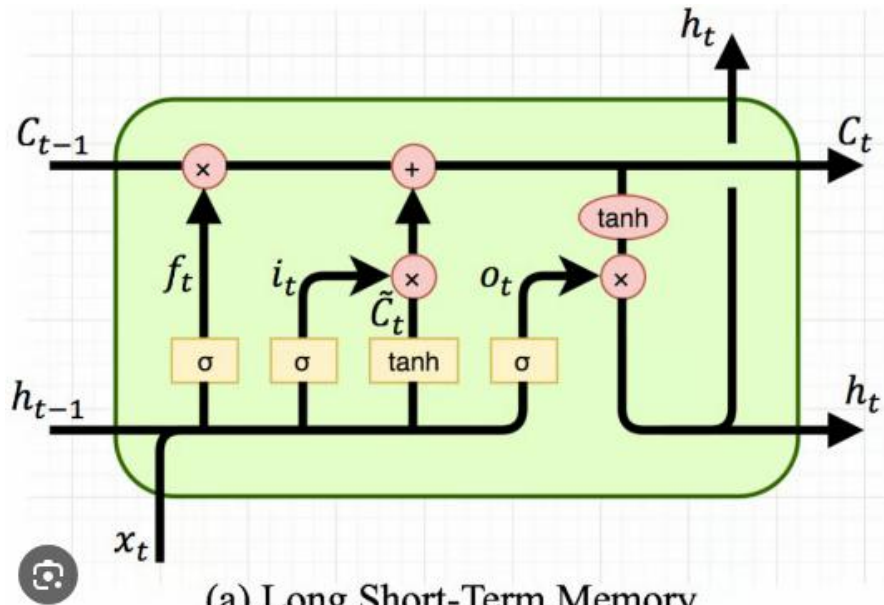
앞부분의 단어 영향이 문장 끝까지 전달되지 못해 **장기 의존성**(long-term dependency)을 학습하기 어렵습니다. 중요 정보와 불필요한 정보를 구분해 관리하는 **LSTM** 같은 개선된 구조의 등장을 이끌었습니다.

4 부: 장단기 기억(LSTM) 네트워크

4.1 정보의 문지기: LSTM 셀 내부 구조



LSTM의 강력함은 더 복잡한 반복 모듈에서 나옵니다. 이 모듈은 은닉 상태 뿐만 아니라, 별도의 셀 상태(cell state)와 이를 제어하는 세 개의 게이트(gate)를 포함하고 있습니다.



(a) Long Short-Term Memory

셀 상태는 LSTM의 핵심입니다. 이는 컨베이어 벨트처럼 직선으로 흐릅니다. 정보는 이 벨트를 따라 거의 변형 없이 전달될 수 있어, 먼 거리의 정보를 보존하기 용이합니다. 이것이 LSTM의 장기 기억에 해당합니다. 셀상태는 장기/은닉층은 단기 기억을 담당합니다.

게이트는 어떤 정보가 중요한지를 학습하는 신경망입니다. 이들은 0과 1 사이의 값을 출력하는 시그모이드(sigmoid) 함수로 구성되어, 각 정보 구성 요소를 얼마나 통과시킬지 결정합니다. '0'은 '아무것도 통과시키지 말라'는 의미이고, '1'은 '모든 것을 통과시키라'는 의미입니다.

- **Forget Gate (망각 게이트, f_t)**

- " C_{t-1} 에 있던 정보 중 **뭘 버릴지?**" 결정.
- 0이면 완전히 잊음, 1이면 그대로 유지.

- **Input Gate (입력 게이트, i_t , $\tilde{C}_t$)**

- i_t : "새로 들어온 정보(x_t, h_{t-1}) 중 **뭘 저장할지?**" 결정

- **Output Gate (출력 게이트, o_t)**

- "지금 뭘 꺼내 쓸까?" 결정.

셀상태와 은닉층을 합쳐 최종 출력(h_t)생성, softmax 같은 분류기를 거쳐 예측

이러한 구조는 기울기 소실 문제를 해결하는 데 결정적인 역할을 합니다. 셀 상태라는 별도의 보호된 정보 고속도로를 두고, 게이트를 통해 (RNN의 반복적인 곱셈 연산이 아닌) 덧셈 방식의 상호작용을 수행함으로써, 기울기가 소실되지 않고 시간을 거슬러 전파될 수 있습니다.

4.3 이론에서 현실로

LSTM은 장기 의존성을 처리하는 능력 덕분에 수년간 다양한 NLP 과제에서 최고의 성능을 보여주었습니다. 대표적인 응용 분야는 다음과 같습니다.

- 기계 번역
- 음성 인식
- 장문의 리뷰에 대한 감성 분석
- 필기체 인식

이 지점에서, 지금까지의 여정을 요약하고 다음 단계로 나아가기 위한 발판을 마련하기 위해 아래의 비교표를 통해 각 모델의 특징을 정리할 수 있습니다. 이 표는 각 모델이 이전 모델의 한계를 어떻게 극복하려 했는지, 그리고 그 과정에서 어떤 새로운 과제에 직면했는지를 명확히 보여줍니다.

모델	핵심 원리	주요 강점	주요 한계
규칙 기반 (예: ELIZA)	수작업으로 만든 언어 규칙과 패턴 매칭.	알려진 패턴에 대해 해석 가능하고 정밀함.	처음 보는 변형에 실패하며, 진정한 이해가 없음.

N-gram (통계)	이전 N-1 개 단어에 기반한 다음 단어의 확률.	단순하고 효율적이며, 노이즈가 있는 데이터에 강건함.	작고 고정된 창 너머의 문맥을 포착할 수 없음.
RNN (순환 신경망)	과거 순서열의 기억 역할을 하는 '은닉 상태' 루프.	이론적으로는 어떤 길이의 의존성이든 모델링 가능.	실제로는 기울기 소실 문제로 인해 단기 기억에 그침.
LSTM (장단기 기억)	정보 흐름을 조절하는 게이트가 있는 셀 상태 (망각, 입력, 출력).	기울기 소실 문제를 해결하여 장기 의존성 학습 가능.	RNN 보다 복잡하고 계산 비용이 높음.

5 부: 거대 언어 모델

5.1 거대 언어 모델(LLM)의 등장

LSTM 은 언어 처리에서 매우 진보한 기술이었습니다. 기억을 오래 유지하고, 이전보다 더 자연스러운 언어 모델을 만들 수 있었습니다.

하지만 긴 문맥을 완전히 이해하기에는 여전히 한계가 있었고, 병렬처리가 어렵다는 점 때문에 대규모 학습에도 제약이 있었습니다.

이러한 한계를 극복하기 위해 등장한 것이 **Transformer**, 그리고 이를 바탕으로 한 **거대 언어모델(LLM)**입니다.”

LLM 은 방대한 양의 텍스트 데이터로 훈련된 매우 큰 딥러닝 모델입니다.

이들은 각 과제에 대한 특정 훈련 없이도 광범위한 작업을 수행할 수 있는 '기본 모델(foundation model)'의 한 종류입니다.

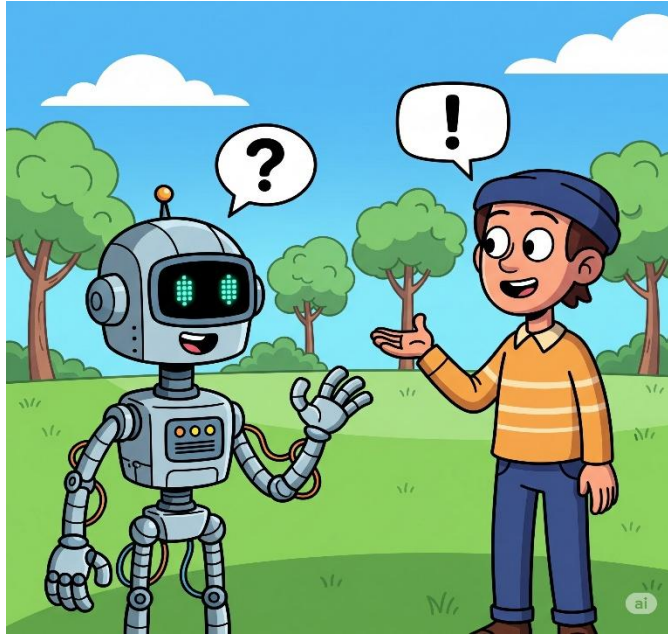
LLM 을 '거대하게' 만드는 요소는 다음과 같습니다.

- **데이터:** 위키피디아, Common Crawl 등 인터넷 규모의 데이터셋으로 훈련되며, 이는 수조 개의 단어로 구성됩니다.
- **파라미터:** 신경망 내의 가중치와 편향을 의미하는 파라미터가 수십억 개에 달합니다.

대부분의 최신 LLM 은 RNN 이나 LSTM 이 아닌, **트랜스포머(Transformer)**라는 새로운 아키텍처에 기반합니다. 트랜스포머는 순차적으로 데이터를 처리하는 RNN 과 달리, '셀프 어텐션(self-attention)'이라는 메커니즘을 사용하여 전체 시퀀스를 병렬로 처리할 수 있습니다. 이를 통해 훨씬 더 큰 데이터셋으로 훈련이 가능하며, 매우 긴 텍스트에 걸친 복잡한 관계를 더 효과적으로 포착할 수 있습니다.

LLM 의 근본적인 작업은 여전히 다음 단어를 예측하는 것이지만, 그 엄청난 규모는 예측을 넘어선 새로운 능력을 발현시켰습니다. LLM 의 근본적인 작업은 여전히 다음 단어를 예측하는 것이지만, 그것을 넘어 대화를 생성하기까지 발전 하였습니다.

5.2 LLM 의 역사 정리



기억을 컴퓨터로

ELIZA : 규칙기반

N-gram : 확률기반

RNN : 순환구조

LSTM : 장기 의존성 극복

LLM : 예측을 넘어 생성

LLM의 근본 원리는 '다음 단어 예측'입니다.

하지만 컴퓨터는 단어 자체를 이해하지 못합니다.

→ 단어를 **숫자의 표현**으로 바꿔야 학습과 예측이 가능합니다.

이 과정을 가능하게 하는 핵심 기술은 **토큰화(Tokenization)**와 **임베딩(Embedding)**입니다.

다음시간에 더 자세하게 배우겠습니다.

참고 자료

1. Information Processing Theory In Psychology - Simply Psychology, 8 월 20, 2025 에 액세스, <https://www.simplypsychology.org/information-processing.html>
2. Your brain does not process information and it is not a computer | Aeon Essays, 8 월 20, 2025 에 액세스, <https://aeon.co/essays/your-brain-does-not-process-information-and-it-is-not-a-computer>
3. What Do You Know: Do Our Minds Work Like Computers?, 8 월 20, 2025 에 액세스, <https://www.td.org/content/atd-blog/what-do-you-know-do-our-minds-work-like-computers>
4. Neuroscience For Kids - brain vs. computer, 8 월 20, 2025 에 액세스, <https://faculty.washington.edu/chudler/bvc.html>
5. What is Natural Language Processing (NLP)? - Talend, 8 월 20, 2025 에 액세스, <https://www.talend.com/resources/what-is-natural-language-processing-nlp/>
6. What is Natural Language Processing (NLP)? | Oracle, 8 월 20, 2025 에 액세스, <https://www.oracle.com/artificial-intelligence/what-is-natural-language-processing/>
7. Natural Language Processing (NLP): What it is and why it matters - SAS, 8 월 20, 2025 에 액세스, https://www.sas.com/en_us/insights/analytics/what-is-natural-language-processing-nlp.html
8. What is Natural Language Processing? - NLP Explained - AWS, 8 월 20, 2025 에 액세스, <https://aws.amazon.com/what-is/nlp/>
9. Master NLP History: From Then to Now - Shelf.io, 8 월 20, 2025 에 액세스, <https://shelf.io/blog/master-nlp-history-from-then-to-now/>
10. The Evolution of Natural Language Processing: From Rule-Based Systems to Deep Learning and Beyond | Simbo AI - Blogs, 8 월 20, 2025 에 액세스, <https://www.simbo.ai/blog/the-evolution-of-natural-language-processing-from-rule-based-systems-to-deep-learning-and-beyond-155891/>
11. A Brief History of Natural Language Processing - DATAVERSITY, 8 월 20, 2025 에 액세스, <https://www.dataversity.net/a-brief-history-of-natural-language-processing-nlp/>

12. What Is An N-Gram? | Coursera, 8 월 20, 2025 에 액세스, <https://www.coursera.org/articles/what-is-n-gram>
13. N-Gram Language Modelling with NLTK - GeeksforGeeks, 8 월 20, 2025 에 액세스, <https://www.geeksforgeeks.org/nlp/n-gram-language-modelling-with-nltk/>
14. What is an N-gram Language model? Explain its working in detail - AIML.com, 8 월 20, 2025 에 액세스, <https://aiml.com/what-is-an-n-gram-model/>
15. What is RNN? - Recurrent Neural Networks Explained - AWS, 8 월 20, 2025 에 액세스, <https://aws.amazon.com/what-is/recurrent-neural-network/>
16. Recurrent neural network - Wikipedia, 8 월 20, 2025 에 액세스, https://en.wikipedia.org/wiki/Recurrent_neural_network
17. Introduction to Recurrent Neural Networks - GeeksforGeeks, 8 월 20, 2025 에 액세스, <https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/>
18. What is Recurrent Neural Networks (RNN)? - Analytics Vidhya, 8 월 20, 2025 에 액세스, <https://www.analyticsvidhya.com/blog/2022/03/a-brief-overview-of-recurrent-neural-networks-rnn/>
19. Long Short-Term Memory (LSTM) | NVIDIA Developer, 8 월 20, 2025 에 액세스, <https://developer.nvidia.com/discover/lstm>
20. What Is Long Short-Term Memory (LSTM)? - MATLAB & Simulink - MathWorks, 8 월 20, 2025 에 액세스, <https://www.mathworks.com/discovery/lstm.html>
21. What is LLM? - Large Language Models Explained - AWS, 8 월 20, 2025 에 액세스, <https://aws.amazon.com/what-is/large-language-model/>
22. What is an LLM (large language model)? - Cloudflare, 8 월 20, 2025 에 액세스, <https://www.cloudflare.com/learning/ai/what-is-large-language-model/>
23. What Are Large Language Models (LLMs)? - IBM, 8 월 20, 2025 에 액세스, <https://www.ibm.com/think/topics/large-language-models>
24. Understanding large language models: A comprehensive guide - Elastic, 8 월 20, 2025 에 액세스, <https://www.elastic.co/what-is/large-language-models>